

Aula Passada

Estrutura de Indexação de Dados

- Introdução
- Índice
- Índice hierárquico
- Árvores de Múltiplos Caminhos
- Árvore B
- Árvore B*
- Árvore B+

Ordenação Interna e Externa

Diego Silva Caldeira Rocha

Sumário

Ordenação Interna e Externa

- Ordenação Interna
- HeapSort
- Ordenção Externa
- MergeSort Externo

Introdução

A	N	A		R	U	A	<i>b</i>	A	U	G	U	S	T	O	<i>b</i>	P	A	I	V	A		I	B	A	T	E		<i>b</i>	<i>b</i>
A	N	T	O	N	I	A		R	U	A	<i>b</i>	X	V	<i>b</i>	D	E	<i>b</i>	M	A	I	O		I	B	A	T	E		<i>b</i>
J	O	A	O		R	U	A	<i>b</i>	A		R	I	O	<i>b</i>	C	L	A	R	O		<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>
M	A	R	I	A		R	U	A	<i>b</i>	1		S	A	O	<i>b</i>	C	A	R	L	O	S		<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>
P	E	D	R	O		R	U	A	<i>b</i>	X	V		S	A	O	<i>b</i>	C	A	R	L	O	S		<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>

Ordenação baseada em um determinado campo, usando suas chaves

Ordenação Interna

- É possível ordenar em memória primária:
 - Etapas:
 - Leitura de todos os registros do arquivo
 - ordenação dos registros em RAM
 - escrita dos registros ordenados no arquivo
 - Custo
 - soma dos custos das 3 etapas
 - Leitura e escrita sequenciais minimizam seeks
 - uso de métodos eficientes de ordenação interna

Heapsort

- Melhora o desempenho da ordenação interna
 - Paraleliza a ordenação com o processamento de
 - entrada e saída (ou seja, leitura e escrita de
 - registros)

parte 1 = leitura; parte 2 = ordenação

parte 1 = ordenação; parte 2 = escrita

possíveis
paralelismos

Heap

Estrutura de Dados: Heap

Heap é definido como uma sequência de elementos com prioridades

$$Pr_1, Pr_2, \dots, Pr_n$$

tal que

$$Pr_i \leq Pr_{\lfloor i/2 \rfloor}, \text{ para } 1 < i \leq n$$

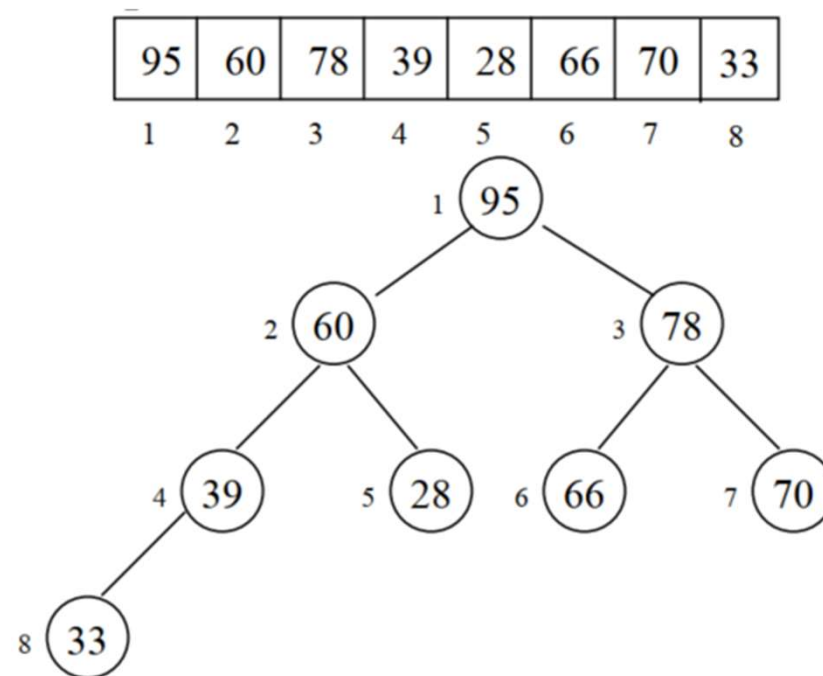
ou

$$\begin{cases} Pr_i \geq Pr_{2i} \\ Pr_i \geq Pr_{2i+1} \end{cases}, \text{ para } 1 \leq i \leq \lfloor n/2 \rfloor$$

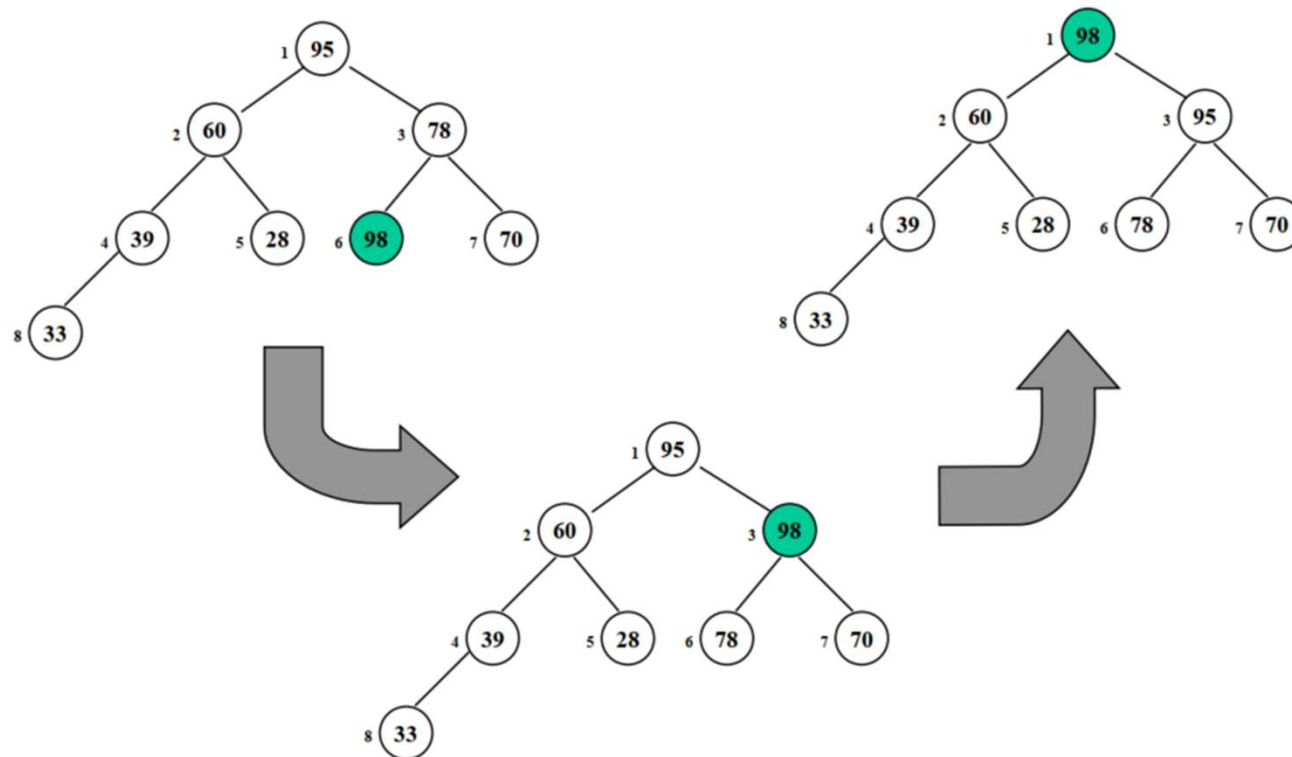
Heap

Heap pode ser visto como uma árvore binária

Exemplo

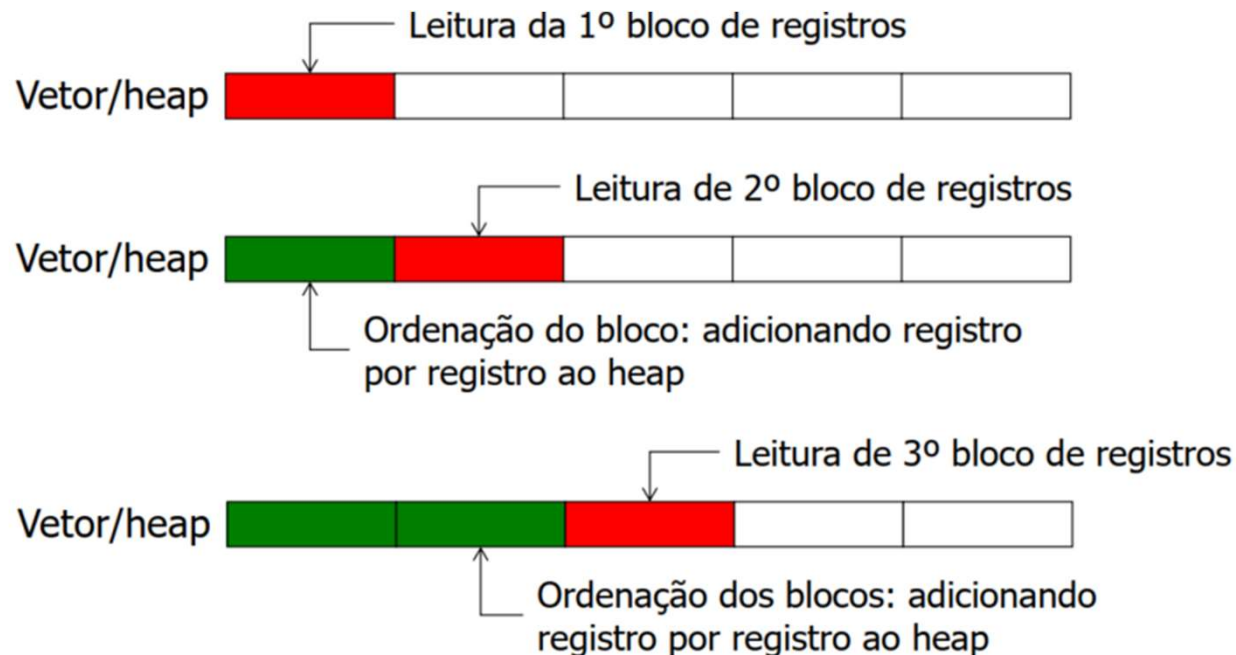


Alteração de Prioridade - Aumento



Ordenação Interna *Heapsort*

- Paralelismo leitura ordenação ou mudança de prioridade

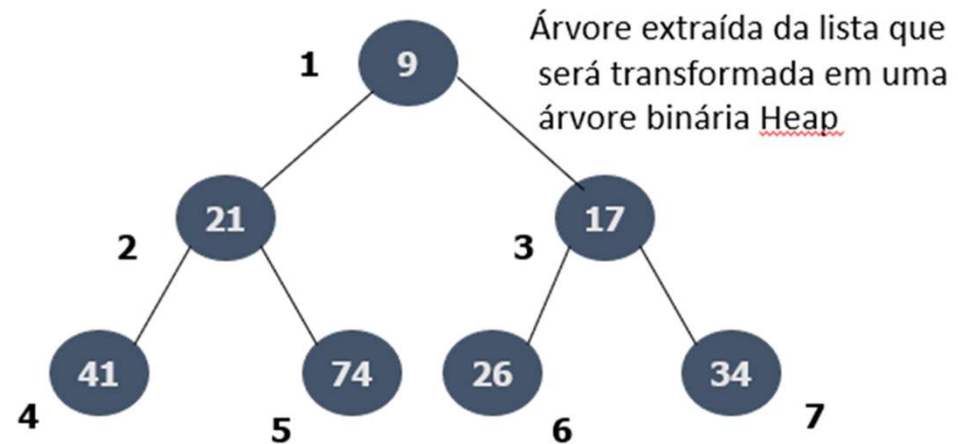


Construindo Heap Invertido

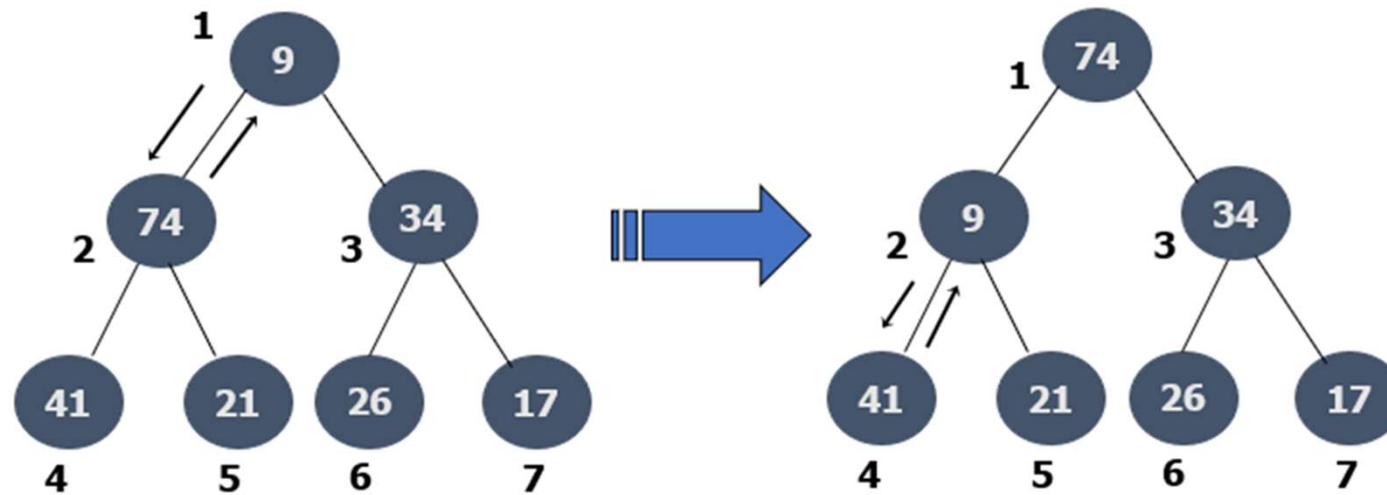
- Método heap-sort:
- Construindo a árvore heap a partir de um vetor:

Vetor ou lista

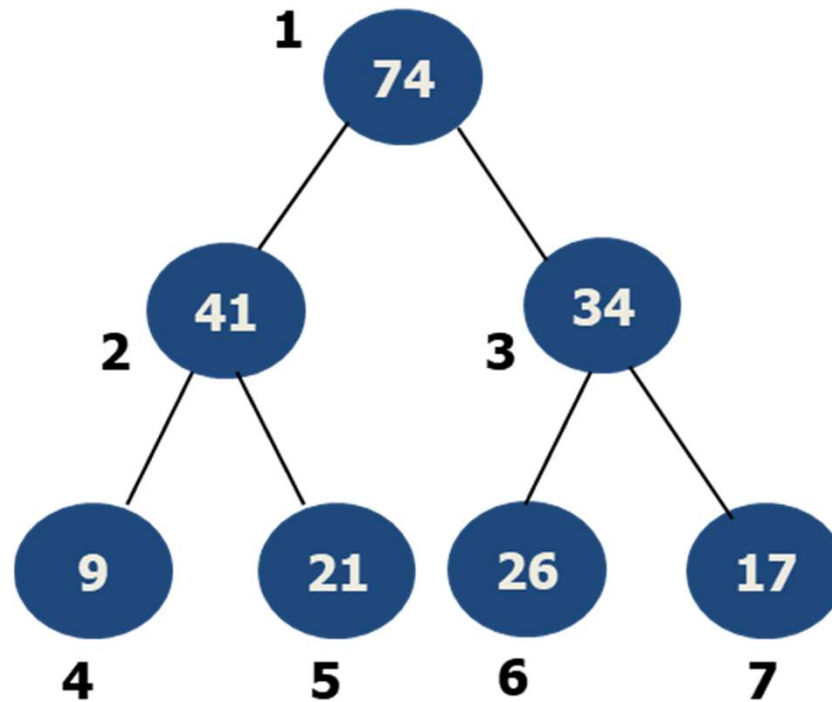
9	21	17	41	74	26	34
1	2	3	4	5	6	7



Construindo o Heap Invertido



Construindo o Heap Invertido



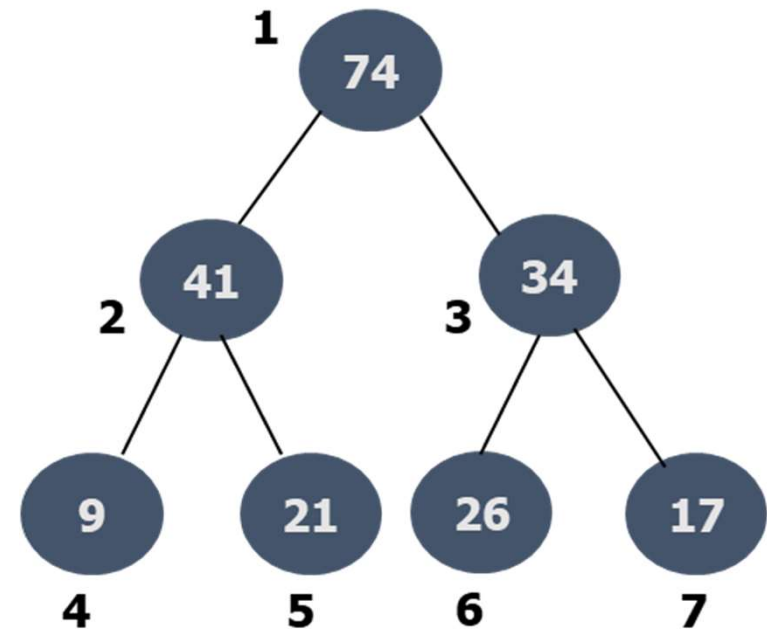
Paralelizando Ordenação com Escrita

Algoritmo:

- recupera o registro da raiz do heap
- enquanto rearranja o heap, grava esse registro no arquivo de saída
- Rearranjo do heap
 - retira o elemento da raiz
 - coloca o último elemento k do heap como raiz
 - enquanto k for maior do que seus filhos, troca-o de lugar com seu menor filho

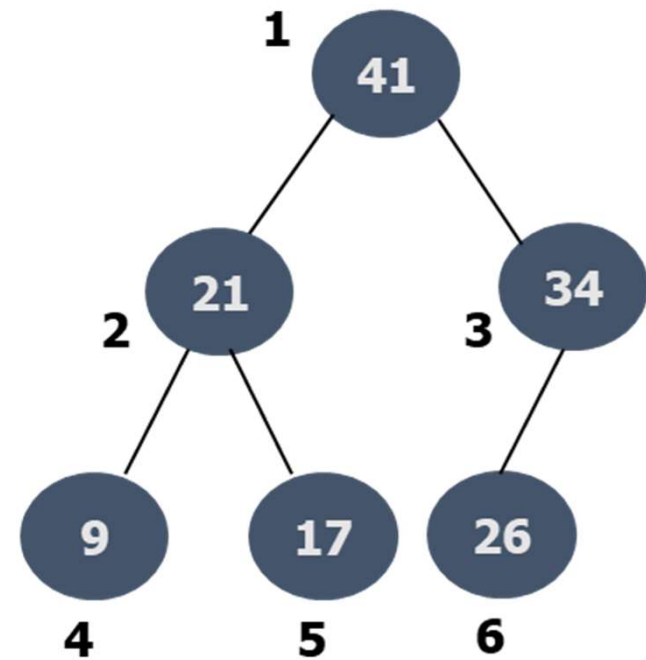
Exemplo

- Recupera-se raiz 74, colocando em seu lugar 17.



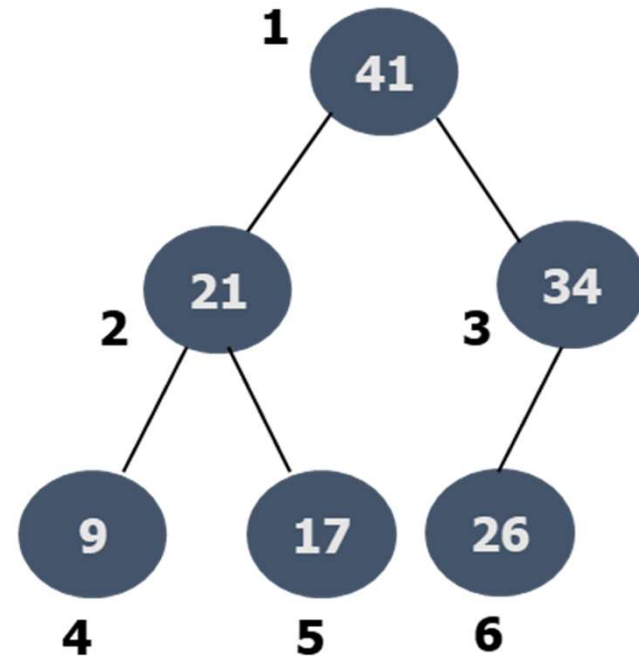
Exemplo

- Enquanto grava o 74 no arquivo ordenado rearranja heap.



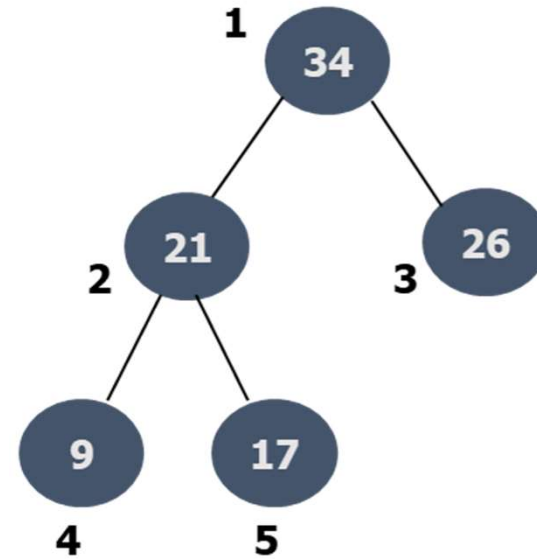
Exemplo

- Recupera a raiz 41 e coloca 26 em seu lugar.



Exemplo

- Enquanto grava o 41 no arquivo ordenado rearranja heap.
- E assim por diante, até *heap* esvaziar



Ordenação Externa

- Os dados são acessados sequencialmente ou em blocos, utilizando tanto a memória principal, quanto o disco para ordená-los, uma vez que a memória principal é insuficiente.

Sort-Merge Externo

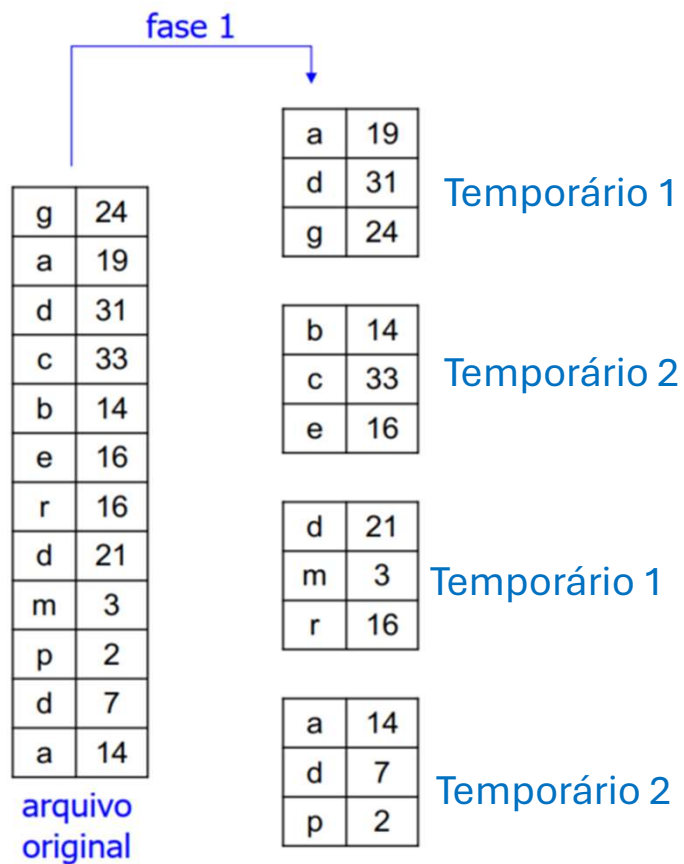
- Fase 1
 - cria subarquivos ordenados (i.e., runs) a partir do arquivo original
- Fase 2
 - combina os subarquivos ordenados em subarquivos ordenados maiores até que o arquivo completo esteja ordenado.

Sort-Merge Externo com Memória 3 registros

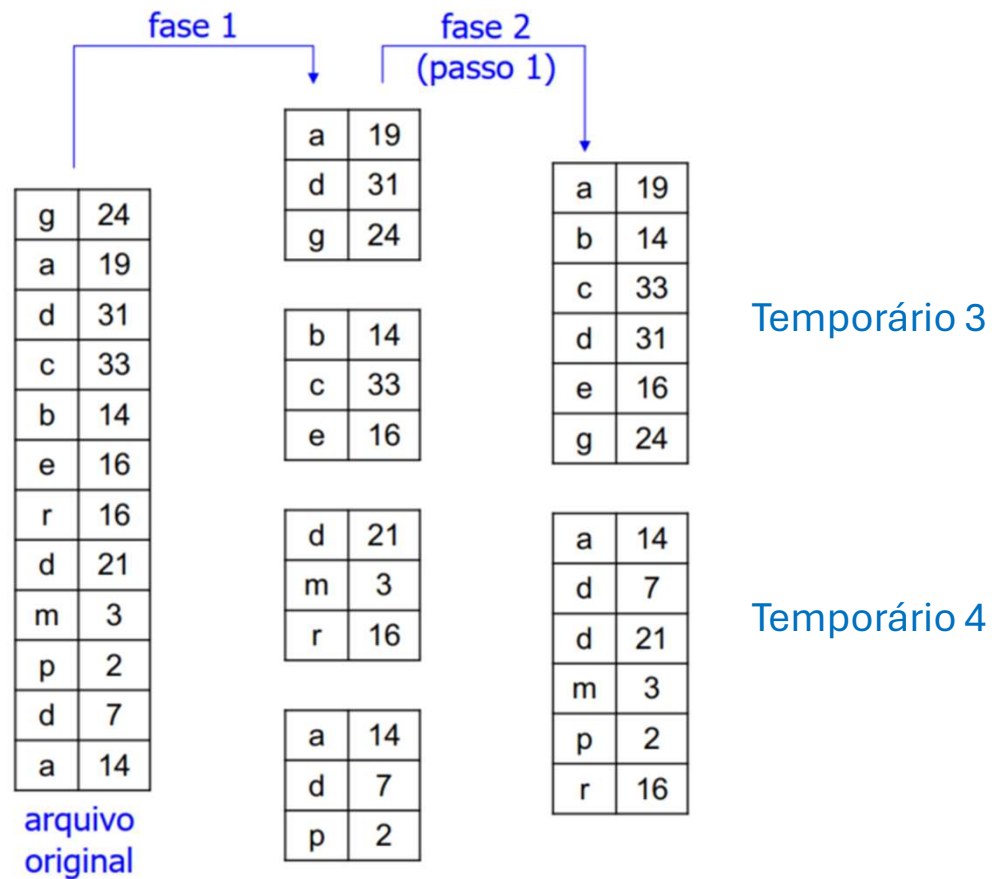
g	24
a	19
d	31
c	33
b	14
e	16
r	16
d	21
m	3
p	2
d	7
a	14

arquivo
original

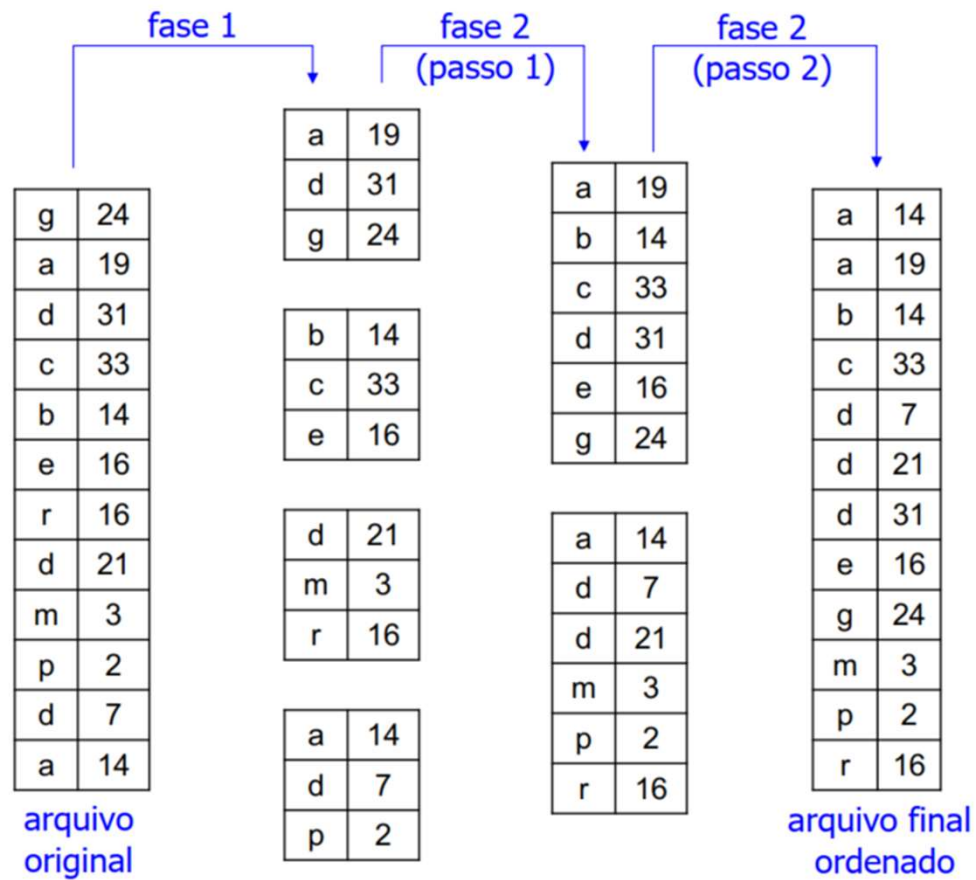
Sort-Merge Externo com Memória 3 registros



Sort-Merge Externo com Memória 3 registros



Sort-Merge Externo com Memória 3 registros



Exercício

1) Considere inserções em uma lista de prioridades utilizando os números: 15,21,23,48,35,70,67

Mostre passo a passo o preenchimento do vetor e na árvore de representação de uma lista. Considerando que a lista representará um Heap Invertido.

Mostre passo a passo, a lista de prioridades do exercício anterior após realizar duas remoções.

Exercício

2) Considerando a capacidade de ordenação em memória de 3 registros, faça a intercalação balanceada de 2 arquivos temporários para os seguintes registros:

C I E N C I A D A C O M P U T A C A O

(obs: mostre todos os passos)

Referência

Szwarcfiter, J.; Markezon, L. Estruturas de Dados e seus Algoritmos.

Próxima aula

Tabela Hash Exteruor.